

Taller CGAL

Estudiantes

Lenin Enrique García Caraballo

David Orlando Riaño Morales

Docente

Alexander Pérez Ruiz

Escuela Colombiana de Ingeniería Julio Garavito

Métodos computacionales en ingeniería – MCEI

2026-2

1 de noviembre de 2026

1 Preguntas de investigación

1.1 ¿Qué es la geometría computacional y qué problemas de ingeniería permite resolver?

La geometría computacional es un área que combina elementos de la geometría clásica euclidiana con la matemática y la informática moderna. Esta área permite resolver problemas cómo:

- Interpolación de funciones sobre cuadrículas irregulares.
- La generación de mallas espaciales.
- Detección de colisiones e insercciones geométricas.

1.1.1. ¿Qué es la geometría computacional (Computational Geometry) y cuáles son sus diferencias con la geometría analítica y la geometría euclidiana tradicional?

Como se mencionó anteriormente, la geometría computacional es un área que combina elementos de la geometría clásica euclidiana con la matemática y la informática moderna. Su principal diferencia con la geometría tradicional, la cual dependía históricamente de igualdades exactas para determinar si un punto estaba dentro, en un borde o en un vértice de una figura, la geometría computacional moderna suele tolerar aproximaciones empleando aritmética de punto flotante para ganar velocidad a la hora de procesar.

1.1.2. ¿Qué tipos de objetos geométricos se utilizan para representar problemas computacionales en dos y tres dimensiones?

Hay varios tipos de objetos geométricos o formas geométricas que pueden usarse para representar problemas bidimensionales y tridimensionales, como por ejemplo, los puntos, las cajas en 2D y en 3D, líneas y segmentos de línea, triángulos, polígonos, círculos y las esferas.

1.1.3. ¿Qué son los puntos, segmentos, polígonos, poliedros, superficies y nubes de puntos desde la perspectiva computacional?

- **Puntos:** Son representaciones de coordenadas cartesianas en cualquier dimensión, almacenadas en código mediante estructuras de números punto flotante y matrices de cualquier tamaño.
- **Segmentos:** Son porciones de una línea acotada entre dos puntos a y b, representadas mediante ecuaciones paramétricas lineales.
- **Polígonos:** Son vectores compuestos por **N** vértices conectados por **N** segmentos de línea dirigidos en un orden cíclico estricto.
- **Poliedros y superficies:** Son volúmenes de tres dimensiones que se descomponen analíticamente calculando determinantes, o superficies en n dimensiones definidas analíticamente por los radios de sus coordenadas.
- **Nubes de puntos:** Son conjuntos de puntos organizados en estructuras de datos jerárquicas para dividir el espacio de manera recursiva, permitiendo ejecutar búsquedas topológicas eficientes.

1.1.4. ¿Cuáles son cinco problemas clásicos de la geometría computacional?

- **Problema del vecino más cercano:** Consiste en localizar rápidamente cual es el punto, dentro de un gran conjunto de datos, se encuentra a menor distancia de una coordenada de referencia empírica o arbitraria. No evalúa iterativamente cada punto, sino que emplea estructuras de partición jerárquica, tal como lo hacen los árboles de decisión, que reducen la carga de trabajo para consultas individuales al descartar ramas enteras del espacio. Alternativamente, para resolver este problema se pueden usar las aristas de una triángulación Delaunay, ya que la conexión más corta entre cualquier punto y su vecino más cercano siempre forma parte de dicha red geométrica.

- **Triangulaciones de Delaunay:** Consiste en conectar puntos formando una red de triángulos que maximizan sus ángulos mínimos, asegurando matemáticamente que la circunferencia circunscrita de cada triángulo no contenga vértices adicionales.

- **Diagrama de Voronoi:** Divide el plano en regiones poligonales donde cada celda engloba el área estrictamente más cercana a un sitio específico, actuando como el grafo dual de la malla de Delaunay.

- **Envolvente convexa:** Consiste en determinar el polígono perimetral continuo más pequeño capaz de encerrar por completo a un conjunto de puntos esparcidos en el plano.

- **Árbol de expansión mínima:** Consiste en trazar la red de segmentos de menor longitud física posible que conecta todos los puntos sin cerrar ningún bucle o ciclo, empleando un subconjunto de aristas de Delaunay.

1.1.5. ¿En qué aplicaciones de ingeniería se utilizan estos algoritmos?

Se aplican en motores de gráficos por computadora y visión artificial, eliminación de polígonos ocultos en el plano visual, cálculos rápidos de varios cuerpos de Coulomb, generación de mallas computacionales y algoritmos de compresión de imágenes. A nivel de control y robótica, son vitales para calcular redes de conexión más económicas, ubicar instalaciones en los mayores espacios vacíos disponibles, y navegar entornos físicos manteniendo la máxima distancia posible respecto a amenazas.

1.1.6. Ejercicio de contextualización

1. **Robótica móvil para planificación de trayectorias y detección de colisiones:** Un robot debe ir de un punto A a un punto B sin chocar con obstáculos. Los obstaculos se modelarían como polígonos en el plano, y el robot se modelaría con un punto. Hay dos problemas involucrados acá: determinar si el segmento de trayectoria del robot intersecta algún borde de un obstáculo y construir un grafo de caminos libres, por ejemplo con una triangulación o con un diagrama de Voronoi, que mantiene la ruta lo más lejos posible de los obstáculos.
   La información de entrada sería un mapa del entorno, obtenido con sensores como LiDAR o cámaras, que se convierte en un conjunto de puntos y en polígonos que representan los obstáculos. También se necesitan la posición actual del robot, el destino y las medidas del robot.

2. **Sistemas de información geográfica para modelado de terreno:** Se conoce la altura del terreno solo en algunos puntos, y se quiere estimar la altura en cualquier otra posición. La solución clásica es triangular los puntos y tratar cada triángulo como una superficie plana. Para una ubicación cualquiera hay que encontrar en qué triángulo cae y calcular la altura por interpolación lineal. La triangulación de Delaunay es útil para esto porque evita triángulos muy delgados y alargados, que dan interpolaciones de mala calidad.

La información de entrada sería un conjunto de puntos con coordenadas (x, y, z). Los datos pueden tomarse de estaciones topográficas, GPS, drones o sensores LiDAR aéreos.

3. **Procesamiento de imágenes médicas para reconstrucción de estructuras anatómicas:** A partir de imágenes de un examen médico se segmenta un órgano o un hueso y se construye un modelo geométrico. El modelo se representa como una malla de triángulos en 3D que aproxima la superficie. Con esa malla se calculan cosas como el volumen, el área superficial o la distancia mínima entre dos estructuras, como por ejemplo, un tumor y un vaso sanguíneo, y se verifica que la malla esté bien formada. Un caso concreto es la envolvente convexa de una estructura, que sirve para medir qué tan irregular o cóncava es su forma.
   En este caso los datos de entrada pueden ser una pila de cortes de imagen 2D, de la que se extraen nubes de puntos 3D sobre el contorno del órgano.

1.2 ¿Cuáles son los principales algoritmos y estructuras de datos de la geometría computacional?

1.2.1. ¿Qué problema resuelve?

- **Convex Hull (envolvente convexa):** Resuelve el problema de encontrar la forma más pequeña que permita encerrar todos los puntos de un conjunto de datos. Permite definir la frontera exterior o el contorno mínimo que envuelve a todos los datos sin incluir espacio vacío innecesario.
- **Delaunay Triangulation:** Consiste en unir un conjunto de puntos para formar una red de triángulos que no se superponen. Busca crear triángulos lo más equilibrados posible, evitando que sean demasiado alargados o estrechos. Además, ningún otro punto debe quedar dentro del círculo que rodea a cada triángulo.
- **Voronoi Diagrams:** Resuelve el problema de dividir un espacio en regiones, donde cada región representa los lugares más cercanos a un punto específico.
- **Nearest Neighbor Search:** Resuelve el problema de encontrar rápidamente los puntos más cercanos a una ubicación determinada dentro de un conjunto de datos.
- **Intersection Detection:** Resuelve el problema de identificar si dos o más objetos geométricos se cruzan, se tocan o se superponen, y determinar dónde ocurre.

1.2.2. ¿Cuál es su principio matemático de funcionamiento?

- **Convex Hull (envolvente convexa):** Se basa en encontrar el conjunto convexo más pequeño que contiene todos los puntos. Utiliza las combinaciones de los puntos para formar una figura en la que cualquier línea trazada entre dos puntos de su interior permanece dentro de la figura.
- **Delaunay Triangulation:** Se basa en el criterio del círculo vacío. Consiste en formar triángulos donde el círculo que pasa por sus tres vértices no contiene ningún otro punto en su interior.
- **Voronoi Diagrams:** El espacio se divide en regiones convexas R_i asociadas a un punto generador p_i, donde cualquier coordenada x dentro de R_i satisface la condición de minimización $d(x, p_i) < d(x, p_j)$ para cualquier otro punto p_j.
- **Nearest Neighbor Search:** Se basa en calcular y comparar las distancias entre un punto de consulta y los demás puntos del conjunto. Busca el punto que tenga la menor distancia,

generalmente utilizando la distancia euclidiana. También puede emplear estructuras como los árboles k-d para acelerar la búsqueda.

- **Intersection Detection:** Se basa en utilizar operaciones matemáticas para determinar si dos objetos geométricos se cruzan o se superponen. Utiliza ecuaciones, determinantes y producto cruz para comprobar la posición de los objetos y determinar si existe una intersección.

1.2.3. ¿Qué algoritmo permite implementarlo?

- **Convex Hull (envolvente convexa):** Se puede implementar mediante algoritmos como Andrew (Monotone Chain), Graham Scan, QuickHull o Jarvis March. Todos permiten encontrar el contorno que encierra un conjunto de puntos.
- **Delaunay Triangulation:** Se puede implementar con el algoritmo Bowyer-Watson, que agrega los puntos uno por uno para formar los triángulos, o mediante métodos de división del conjunto y unión de resultados (Divide and Conquer).
- **Voronoi Diagrams:** Se pueden generar con el algoritmo de Fortune, que divide el espacio utilizando una línea de barrido. También se pueden construir a partir de una triangulación de Delaunay.
- **Nearest Neighbor Search:** Se puede implementar usando estructuras como los árboles k-d (k-d trees), que organizan los puntos para facilitar la búsqueda, o los Octrees, utilizados para organizar datos en espacios tridimensionales.
- **Intersection Detection:** Se emplea el algoritmo Bentley-Ottmann, que detecta cruces entre varios segmentos. Para objetos tridimensionales se utilizan estructuras como BVH y algoritmos como GJK, que permiten detectar colisiones entre objetos.

1.2.4. ¿Cuál es su complejidad computacional? Indique los supuestos y el tamaño de entrada al que se refiere.

- **Convex Hull (envolvente convexa):** Tiene complejidad $O(n \log n)$ donde n representa la cantidad de puntos en un plano 2D. Esta complejidad se debe principalmente a que primero es necesario ordenar los puntos por sus coordenadas.
- **Delaunay Triangulation:** Tiene complejidad $O(n \log n)$ donde n representa la cantidad de puntos que se quieren conectar. Se supone que los puntos están distribuidos de manera general.
- **Voronoi Diagrams:** Tiene complejidad $O(n \log n)$ donde n representa la cantidad de puntos que generan las regiones. Se supone que el espacio es bidimensional (2D) y que el número de divisiones del diagrama crece proporcionalmente a la cantidad de puntos.
- **Nearest Neighbor Search:** Tiene complejidad $O(n \log n)$ para construir la estructura de búsqueda y $O(\log n)$ para consultar un punto en promedio. n representa la cantidad total de puntos almacenados. Se supone que se utiliza un árbol k-d equilibrado y que los datos tienen pocas dimensiones (2D o 3D). En el peor caso, la búsqueda puede llegar a $O(n)$.
- **Intersection Detection:** Tiene complejidad $O((n+k) \log n)$ donde n representa la cantidad de segmentos que se analizan y k la cantidad de intersecciones encontradas. Se supone que se utiliza el algoritmo Bentley-Ottmann y que comprobar si dos segmentos se cruzan tiene un costo constante.

1.2.5. ¿Qué aplicación concreta podría tener en ingeniería?

- **Convex Hull (envolvente convexa):** En robótica y diseño mecánico, se utiliza para crear contornos alrededor de piezas o robots. Esto permite detectar posibles choques con otros objetos de manera más rápida y facilitar la planificación de movimientos.
- **Delaunay Triangulation:** En redes de sensores IoT, permite conectar los nodos cercanos para analizar cómo se comunican entre sí. También se utiliza en ingeniería civil para crear modelos de terrenos y mapas en 3D.
- **Voronoi Diagrams:** En telecomunicaciones y redes IoT, permite determinar el área que cubre cada antena o sensor. Así se pueden identificar zonas sin cobertura o lugares que necesitan nuevos dispositivos.
- **Nearest Neighbor Search:** En robótica y visión artificial, permite identificar los puntos más cercanos dentro de la información obtenida por cámaras o sensores LiDAR. Esto ayuda a reconocer objetos, detectar obstáculos y construir modelos 3D del entorno.
- **Intersection Detection:** En robótica, permite detectar si un brazo robótico o un robot móvil puede chocar con algún objeto durante su movimiento. En diseño mecánico, ayuda a comprobar si las piezas de una máquina encajan correctamente antes de fabricarlas.

1.2.6. Ejercicio de contextualización

Diagramas de Voronoi:

Se dividiría la región en 100 zonas, una para cada sensor. Cada zona estaría formada por los lugares que se encuentran más cerca de ese sensor que de los demás.

Se identificarían las zonas con áreas muy grandes, ya que podrían indicar lugares donde hay pocos sensores y sería necesario instalar nuevos dispositivos.

Las zonas muy pequeñas indicarían que hay varios sensores concentrados en una misma área. Esto permitiría detectar posibles redundancias y reorganizar su distribución.

Triangulación de Delaunay:

Se conectarían los sensores usando líneas para formar una red de triángulos. Esto permitiría identificar cuáles sensores son vecinos y están más próximos entre sí.

Las conexiones servirían para realizar las rutas de comunicación entre los dispositivos y organizar la red.

Si un sensor deja de funcionar, se podrían recalcular las conexiones entre los sensores cercanos para mantener la comunicación y evitar que la red quede incomunicada.

Los diagramas de Voronoi permitirían analizar cómo se distribuyen los sensores y qué zonas cubre cada uno, mientras que la triangulación de Delaunay permitiría identificar las conexiones y relaciones de proximidad entre ellos.

Al implementar un programa en C++ para trabajar con los datos del archivo ubicaciones_100_sensores_xy, se obtuvieron los siguientes resultados:

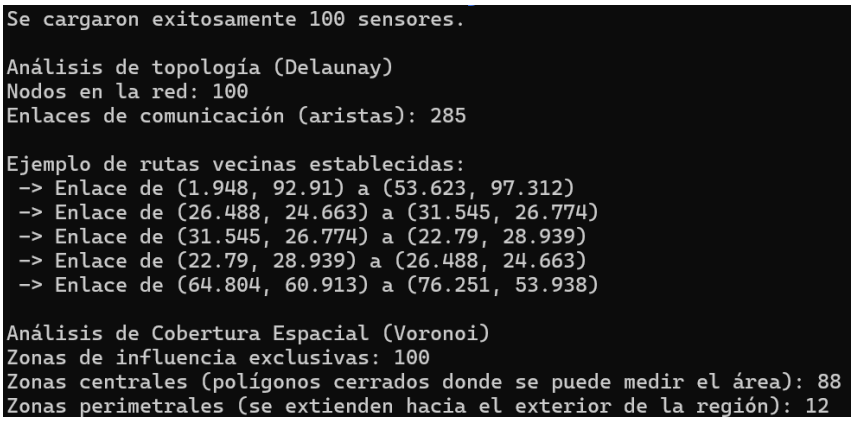


Figura 1: Resultados de implementación de ejercicio de contextualización de parte 2 en C++

El algoritmo identificó los 100 sensores y generó 285 conexiones entre ellos. Estas conexiones permiten representar las relaciones de proximidad entre los dispositivos mediante una red de triángulos, evitando que las aristas se crucen, y así, identificar qué sensores son vecinos y cuáles podrían establecer enlaces de comunicación. Aunque Delaunay genera una red que favorece las conexiones entre puntos cercanos, no busca minimizar el número total de enlaces, sino construir una triangulación que cumpla sus propiedades geométricas.

Las 88 zonas centrales, son los sensores ubicados en el interior de la región. Sus áreas de cobertura están limitadas por la presencia de los sensores vecinos. Un polígono muy grande indica que falta cobertura, mientras que polígonos pequeños indica que hay sensores redundantes.

Las 12 zonas perimetrales representan los 12 sensores que conforman la frontera exterior de la red. Son zonas no acotadas que se extienden indefinidamente hacia el exterior. En la práctica, estos 12 nodos son los encargados de delimitar la frontera física del sistema.

1.3 ¿Por qué la precisión numérica y la robustez son desafíos fundamentales en geometría computacional?

1.3.1. ¿Qué problemas introduce la representación numérica IEEE-754 en los cálculos geométricos?

Introduce errores de aproximación y redondeo, ya que los computadores utilizan una cantidad limitada de bits para representar números decimales, es decir, algunos valores no se pueden almacenar de forma exacta, lo que afecta los cálculos que se realicen.

Por ejemplo, al calcular si tres puntos están alineados o si dos líneas se cruzan, el redondeo puede cambiar el resultado y hacer que el programa interprete incorrectamente la posición de los objetos.

En figuras geométricas, los errores acumulados durante los cálculos pueden provocar que se formen triángulos incorrectos, que algunas figuras se superpongan o que el programa detecte intersecciones donde realmente no existen.

1.3.2. ¿Qué significa que tres puntos sean colineales y por qué determinarlo computacionalmente puede ser difícil?

Tres puntos son colineales cuando se encuentran sobre una misma línea recta. Esto significa que no forman un triángulo con área, ya que los tres puntos están alineados. Se puede comprobar

calculando un determinante con las coordenadas de los tres puntos: si el resultado es cero, son colineales.

Determinar esta condición en un computador puede ser difícil porque se utilizan números de precisión limitada, lo que genera errores de redondeo.

Esto puede provocar que el programa interprete incorrectamente los puntos, considerando que no están alineados cuando realmente sí lo están. Algunos algoritmos geométricos, como la envolvente convexa o la triangulación de Delaunay, pueden generar resultados incorrectos o presentar fallos durante su ejecución.

1.3.3. ¿Qué son las configuraciones degeneradas (degenerate cases)?

Son situaciones en las que los puntos de entrada tienen una disposición geométrica que puede dificultar el funcionamiento normal de los algoritmos.

Algunos ejemplos son los puntos colineales o repetidos, que ocurren cuando varios puntos están sobre una misma línea recta o tienen las mismas coordenadas. En estos casos, una figura como la envolvente convexa podría dejar de formar un polígono y convertirse en una línea o incluso en un solo punto.

Otro ejemplo son los puntos cocirculares, que aparecen cuando cuatro o más puntos se encuentran sobre la circunferencia de un mismo círculo. Esto puede generar problemas en la triangulación de Delaunay, ya que pueden existir diferentes formas de conectar los puntos que cumplen las mismas condiciones geométricas.

Estas situaciones son difíciles de manejar computacionalmente porque requieren cálculos muy precisos para distinguir cuándo un resultado es exactamente cero. Los errores de redondeo pueden provocar que los algoritmos tomen decisiones equivocadas y generen resultados incorrectos o fallen durante su ejecución.

1.3.4. ¿Cómo pueden los errores de redondeo alterar la detección de intersecciones, la orientación de puntos o la construcción de triangulaciones?

Los errores de redondeo pueden alterar los cálculos geométricos porque los computadores no representan todos los números con exactitud. Esto puede provocar que los algoritmos interpreten incorrectamente la posición de los puntos o la relación entre diferentes figuras.

En la detección de intersecciones, estos errores pueden hacer que el programa detecte un cruce entre dos objetos que realmente no se tocan o que ignore una intersección que sí existe. Las coordenadas calculadas pueden tener pequeñas diferencias respecto a sus valores reales.

En la orientación de puntos, los errores de redondeo pueden cambiar el resultado al determinar si un punto está a la izquierda, a la derecha o sobre una línea recta. Por ejemplo, un pequeño error puede hacer que el programa interprete que un punto está en el lado contrario o que no está alineado con otros dos.

En la construcción de triangulaciones, estos errores pueden provocar que los algoritmos conecten incorrectamente los puntos, generando triángulos superpuestos o líneas que se cruzan.

1.3.5. ¿Qué diferencia existe entre exactitud geométrica, precisión numérica y robustez algorítmica?

- **Precisión numérica:** Es qué tan bien puede una computadora representar los números. Como utiliza una cantidad limitada de bits para almacenar los valores, pueden aparecer errores de redondeo. Tener una alta precisión numérica no significa que los resultados sean completamente exactos.
- **Exactitud geométrica:** Se refiere a que los cálculos respeten correctamente las relaciones geométricas entre los puntos y las figuras. Por ejemplo, determinar si tres puntos están alineados o si un punto se encuentra a la izquierda o a la derecha de una línea.
- **Robustez algorítmica:** Es la capacidad de un algoritmo para funcionar correctamente incluso cuando se presentan errores de redondeo o situaciones geométricas especiales, como puntos repetidos o alineados.

1.3.6. Experimento conceptual

Si ϵ tiene el valor de 0, el punto C se ubica en $(2, 2)$, por lo que A, B y C se encuentran sobre la recta $y = x$ y son matemáticamente colineales. Si ϵ tiene cualquier otro valor (así sea muy pequeño), los puntos forman un triángulo y dejan de ser colineales.

Computacionalmente, pueden presentarse problemas cuando se utilizan números de punto flotante, como los del estándar IEEE-754. Estos números tienen una cantidad limitada de bits para representar los valores. Si ϵ es muy pequeño en comparación con la precisión disponible, $2 + \epsilon$ puede redondearse a 2.0. En ese caso, el programa no podrá distinguir la pequeña desviación del punto C y calculará un determinante igual a cero, clasificando incorrectamente los puntos como colineales, aunque matemáticamente no lo sean. También puede ocurrir lo contrario, si los tres puntos son colineales, pero sus coordenadas pasan por operaciones, los errores de redondeo pueden acumularse. Esto puede dar un determinante cercano a 0, haciendo que el algoritmo clasifique los puntos como no colineales, aunque matemáticamente los puntos continúen alineados.

Para evitar estos errores, la librería CGAL separa las operaciones en 2 categorías: predicados (predicates) y construcciones (constructions). Estas operaciones se gestionan mediante estructuras llamadas Geometry Kernels, que permiten seleccionar diferentes niveles de precisión y rendimiento.

Predicados: Funciones geométricas que evalúan un conjunto de coordenadas y devuelven un valor discreto o booleano (por ejemplo: ¿están colineales?, ¿gira a la izquierda?, ¿está el punto dentro del círculo?).

Construcciones: Son operaciones matemáticas que toman geometrías existentes y calculan objetos o coordenadas nuevas (por ejemplo: el punto exacto donde se cruzan dos rectas, o el centro de gravedad de un polígono).

Los distintos kernels de CGAL permiten al programador elegir el nivel de exactitud deseado para optimizar el rendimiento:

Predicados exactos: Garantizan que cualquier decisión espacial del algoritmo sea totalmente correcta frente a la geometría matemática real.

Construcciones exactas: Garantizan que las nuevas coordenadas y geometrías calculadas se obtengan sin perder exactitud durante las operaciones matemáticas. Esto es útil cuando se

realizan varias operaciones consecutivas, pero exige un alto consumo de memoria y tiempo de procesamiento.

No todos los kernels de CGAL ofrecen ambas propiedades al mismo tiempo. Por ejemplo, Exact_predicates_inexact_constructions_kernel garantiza predicados exactos, pero utiliza cálculos aproximados para generar nuevas coordenadas. Exact_predicates_exact_constructions_kernel ofrece tanto predicados como construcciones exactas, útil cuando se necesita mayor exactitud, pero tiene mayor costo computacional.

1.4 ¿Qué es CGAL, cómo está organizada y qué ventajas ofrece frente a una implementación propia?

1.4.1. ¿Qué significa CGAL y cuál es el propósito de la librería?

CGAL significa Computational Geometry Algorithms Library (Biblioteca de Algoritmos de Geometría Computacional). Su propósito es ofrecer herramientas para resolver problemas de geometría computacional. Permite realizar operaciones geométricas, como calcular intersecciones, construir triangulaciones, identificar puntos cercanos y analizar relaciones espaciales, procurando evitar errores numéricos que puedan afectar los resultados.

Facilita el trabajo de ingenieros y desarrolladores, ya que proporciona algoritmos que pueden utilizarse y adaptarse según las necesidades de cada proyecto, sin tener que programarlos desde cero.

1.4.2. ¿Por qué está implementada principalmente en C++?

Porque los algoritmos de geometría computacional necesitan realizar muchos cálculos y trabajar con estructuras complejas, como grandes conjuntos de puntos. C++ es un lenguaje compilado que ofrece un alto rendimiento y permite ejecutar estas operaciones en tiempos razonables.

C++ permite utilizar la programación genérica mediante plantillas (templates). Esto permite desarrollar un algoritmo que pueda funcionar con diferentes tipos de datos y niveles de precisión, sin tener que modificar su implementación. Por ejemplo, un mismo algoritmo de triangulación puede utilizar distintos Geometry Kernels según las necesidades de exactitud del proyecto.

C++ también permite tener un mayor control sobre las operaciones matemáticas y los tipos de datos utilizados. Puede sobrecargar operadores, para usar bibliotecas capaces de trabajar con números de precisión arbitraria. Esto es importante para implementar cálculos exactos en CGAL y reducir los problemas causados por los errores de redondeo del estándar IEEE-754.

1.4.3. ¿Qué es un Geometry Kernel y qué funciones desempeña?

Un Geometry Kernel es uno de los componentes principales de la librería CGAL. Se encarga de definir cómo se representan y se calculan los elementos geométricos, y funciona como una conexión entre los algoritmos de geometría computacional y las operaciones matemáticas que estos necesitan. Establece las herramientas y las reglas que utilizarán los algoritmos para trabajar con figuras y coordenadas en dos o tres dimensiones.

Una de sus funciones principales es definir los elementos geométricos básicos que se utilizan en los cálculos, como puntos (Point_2), líneas, segmentos y vectores. Estos elementos permiten representar objetos y posiciones dentro de un espacio

También se encarga de realizar los predicados (predicates), que son operaciones que permiten determinar relaciones entre los elementos geométricos. Por ejemplo, comprobar si tres puntos están alineados, si un punto se encuentra a la izquierda o a la derecha de una línea, o si dos figuras se intersectan.

Otra función es realizar las construcciones (constructions), que consisten en calcular nuevos elementos geométricos a partir de otros existentes. Por ejemplo, encontrar las coordenadas del punto donde se cruzan dos líneas o calcular una nueva posición dentro de una figura.

También permite controlar la precisión numérica y la robustez de los cálculos. Dependiendo del kernel seleccionado, las operaciones pueden realizarse con cálculos aproximados, que suelen ser más rápidos, o mediante cálculos exactos, que ofrecen mayor confiabilidad.

1.4.4. ¿Qué son los predicates y las constructions?

Predicados: Funciones geométricas que evalúan un conjunto de coordenadas y devuelven un valor discreto o booleano (por ejemplo: ¿están colineales?, ¿gira a la izquierda?, ¿está el punto dentro del círculo?).

Construcciones: Son operaciones matemáticas que toman geometrías existentes y calculan objetos o coordenadas nuevas (por ejemplo: el punto exacto donde se cruzan dos rectas, o el centro de gravedad de un polígono).

1.4.5. ¿Qué diferencias existen entre Exact_predicates_inexact_constructions_kernel y Exact_predicates_exact_constructions_kernel?

Exact_predicates_inexact_constructions_kernel: Garantiza que los predicados, se realicen de manera exacta. Sin embargo, cuando necesita calcular nuevas coordenadas o elementos geométricos, utiliza operaciones aproximadas mediante números de punto flotante.

Es una de las opciones más utilizadas porque ofrece un buen equilibrio entre precisión y rendimiento. Permite que las decisiones geométricas sean confiables y mantiene una buena velocidad de procesamiento al utilizar cálculos aproximados para generar nuevas geometrías.
Exact_predicates_exact_constructions_kernel: Garantiza tanto la exactitud de las decisiones geométricas (predicados) como la de los nuevos elementos que se calculan (construcciones). Evita los errores de redondeo durante las operaciones y reduce la posibilidad de que estos errores se acumulen y afecten los resultados posteriores.

Da mayor exactitud en todos los cálculos geométricos, pero, requiere más recursos computacionales.

1.4.6. ¿Qué ventajas y limitaciones tiene utilizar CGAL frente a implementar directamente los algoritmos?

Ventajas:

CGAL cuenta con herramientas que permiten reducir los problemas causados por los errores de redondeo y la precisión limitada de los computadores.

Al utilizar CGAL, el desarrollador no necesita implementar todos los algoritmos y operaciones matemáticas desde cero, ahorrando tiempo.

CGAL ofrece algoritmos y estructuras de datos que han sido desarrollados y optimizados para resolver diferentes problemas de geometría computacional. Esto permite obtener resultados eficientes y confiables, sin tener que invertir tiempo en optimizar una implementación propia

Desventajas:

Aunque CGAL facilita la implementación de algoritmos geométricos, es necesario comprender cómo funciona su arquitectura. Por ejemplo, el desarrollador debe aprender a seleccionar y configurar los Geometry Kernels, entendiendo las diferencias entre los cálculos exactos y aproximados para elegir la opción más adecuada según el proyecto.

La librería requiere configurar un entorno de desarrollo y gestionar dependencias para poder utilizarla. Implica trabajar con herramientas como CMake, configurar las rutas de las bibliotecas y vincular correctamente los componentes necesarios. Para problemas geométricos sencillos, este proceso puede ser más complicado y tomar más tiempo que implementar directamente un algoritmo pequeño.

1.4.7. Ejercicio de contextualización

2D Triangulations (Triangulaciones en 2D):

- Problema que resuelve: Permite organizar conjuntos de puntos distribuidos en un plano, identificando sus relaciones espaciales y de proximidad. También permite dividir una región en triángulos y determinar la ubicación de puntos dentro de esa distribución.

- Principales tipos de datos y algoritmos: CGAL::Delaunay_triangulation_2, que permite construir triangulaciones de Delaunay, y CGAL::Constrained_Delaunay_triangulation_2, que permite generar triangulaciones respetando segmentos o límites definidos previamente. También permite realizar búsquedas de vecinos cercanos y construir diagramas de Voronoi.

- Posible aplicación: Se puede utilizar para estudiar la distribución de sensores en una región, como en el ejercicio de los 100 sensores. También permite identificar zonas con baja concentración de dispositivos, establecer relaciones de proximidad y planificar posibles conexiones entre ellos.

AABB Tree (Árbol de cajas delimitadoras):

- Problema que resuelve: Resuelve el problema de identificar de manera eficiente si dos objetos geométricos se intersectan o qué tan cerca se encuentran. Es útil cuando se trabaja con modelos tridimensionales que contienen muchos elementos, como piezas mecánicas, estructuras, superficies o modelos de un entorno. Organiza los objetos en una estructura jerárquica de cajas imaginarias que los contienen. Primero comprueba si la trayectoria u objeto de consulta atraviesa una de esas cajas. Si no la atraviesa, descarta todos los elementos que contiene sin revisarlos individualmente. Si sí la atraviesa, continúa examinando las cajas más pequeñas hasta identificar los objetos con los que podría existir una intersección.

- Principales tipos de datos y algoritmos: Su estructura principal es CGAL::AABB_tree, que permite almacenar objetos geométricos como triángulos y segmentos. Tiene operaciones para comprobar si un objeto intersecta con otros, identificar los elementos que presentan intersecciones y encontrar el punto más cercano a una posición determinada.

- Posible aplicación: Un ejemplo puede ser al diseñar piezas mecánicas, para comprobar que 2 piezas deban encajar sin superponerse, o en robótica, para comprobar si el movimiento de un robot puede chocar contra algún objeto.

3D Boolean Operations on Nef Polyhedra (Operaciones booleanas 3D):

- Problema que resuelve: Permite combinar, recortar o comparar objetos tridimensionales mediante operaciones matemáticas conocidas como operaciones booleanas. Su propósito es facilitar la modificación de modelos 3D cuando se necesita unir varias piezas, quitar una parte de un objeto o encontrar el volumen que comparten dos figuras.

- Principales tipos de datos y algoritmos: Su estructura principal es CGAL::Nef_polyhedron_3, que permite representar objetos tridimensionales y realizar las operaciones booleanas sobre ellos.

- Posible aplicación: Por ejemplo, crear diseños 3D, se pueden unir distintas figuras para obtener la geometría deseada o restar una figura de otra para generar perforaciones.

Point Set Processing (Procesamiento de nubes de puntos):

- Problema que resuelve: Permite trabajar con nubes de puntos 3D, que son conjuntos de muchos puntos que representan la superficie de un objeto. Estos puntos normalmente se obtienen mediante escáneres 3D, sensores LiDAR, cámaras de profundidad o sistemas de fotogrametría. Puede ayudar a limpiar el ruido de los sensores y a procesar la información.

- Principales tipos de datos y algoritmos: Trabaja principalmente con puntos tridimensionales representados mediante CGAL::Point_3 y vectores que representan las normales de las superficies, como CGAL::Vector_3. Entre sus funciones están: CGAL::remove_outliers(): identifica y elimina puntos aislados, CGAL::grid_simplify_point_set(): reduce la cantidad de puntos agrupándolos según una cuadrícula, CGAL::jet_smooth_point_set(): suaviza las posiciones de los puntos para reducir el ruido, CGAL::jet_estimate_normals(): calcula las normales de las superficies, CGAL::compute_average_spacing(): calcula la distancia promedio entre los puntos y sus vecinos.

- Posible aplicación: En inspección de estructuras con drones con sensores LiDAR, se pueden eliminar mediciones incorrectas y reducir la cantidad de puntos para construir los modelos. En ingeniería civil se puede usar para procesar las mediciones tridimensionales de los terrenos.

dD Spatial Searching (Búsqueda espacial en varias dimensiones)

- Problema que resuelve: Resuelve el problema de buscar elementos cercanos o ubicados dentro de una región específica cuando se tiene una gran cantidad de puntos almacenados. Su objetivo es encontrar los elementos más cercanos o más lejanos a una posición determinada, así como identificar cuáles se encuentran dentro de una región específica.

- Principales tipos de datos y algoritmos: Utiliza estructuras de datos como CGAL::Kd_tree, que organiza los puntos con divisiones del espacio para facilitar las búsquedas. Tiene funciones como CGAL::K_neighbor_search, que permite encontrar los K vecinos más cercanos, y CGAL::Incremental_neighbor_search, que permite obtener los vecinos de manera progresiva. Incluye búsquedas exactas y aproximadas dentro de regiones definidas mediante elementos como Fuzzy_sphere y Fuzzy_iso_box.

- Posible aplicación: Podría usarse en carros autónomos para identificar los obstáculos más

cercanos a partir de una nube de puntos obtenida por un sensor LiDAR.

1.5 ¿Cómo se puede utilizar CGAL para resolver problemas reales de ingeniería?

1.5.1. ¿Cuál es el problema de ingeniería?

- **Agricultura de precisión:** Obtener modelos tridimensionales del terreno y de los cultivos a partir de las mediciones realizadas por drones con sensores LiDAR. Se recopilan muchos puntos que representan la superficie. Se debe procesar esa información para eliminar errores y construir modelos que permitan analizar las características del terreno e identificar cambios en los cultivos.
- **Diseño mecánico (CAD):** Crear y modificar piezas mecánicas con modelos tridimensionales. Se necesitan unir piezas, realizar perforaciones, quitar material y comprobar cómo encajan diferentes componentes.
- **Ingeniería civil:** Procesar las mediciones tridimensionales obtenidas con sensores LiDAR, drones y equipos de escaneo topográfico para construir modelos digitales de terrenos y estructuras. Estos datos permiten analizar elevaciones, pendientes y desniveles, calcular movimientos de tierra y estudiar posibles deformaciones en las construcciones.
- **Robótica:** Lograr que un robot pueda desplazarse de manera segura sin chocar con los objetos que encuentra a su alrededor. Se deben identificar los obstáculos, calcular rutas de movimiento y comprobar que estas no atraviesen objetos.

1.5.2. ¿Qué información geométrica se necesita?

- **Agricultura de precisión:** Se necesita una gran cantidad de puntos en tres dimensiones, cada uno con sus coordenadas X, Y y Z, que representan la forma y la posición del terreno y los cultivos. Estos datos normalmente se obtienen con sensores LiDAR o fotografías tomadas por drones. También se pueden calcular vectores normales para cada punto, que permiten conocer la orientación de la superficie.
- **Diseño mecánico (CAD):** Se necesitan modelos tridimensionales de las piezas, representados mediante puntos, líneas y superficies que forman sus límites. También se requiere conocer las dimensiones y características de los objetos, como el radio de cilindros y esferas, la ubicación de sus centros y las direcciones en las que se deben extender.
- **Ingeniería civil:** Se necesitan las coordenadas de diferentes puntos del terreno, su altura, para representar las elevaciones y desniveles de una zona. También se deben incluir líneas que indiquen cambios importantes en el terreno, como los bordes de ríos o las crestas de montañas. Se necesitan polígonos que delimiten el área donde se desea construir el modelo.
- **Robótica:** Se necesita información sobre la geometría del robot, sus dimensiones y formas de sus diferentes partes móviles. Para simplificar los cálculos, estas partes pueden representarse con figuras geométricas como cajas o cilindros. También se necesita una representación del entorno donde se mueve el robot, incluyendo los obstáculos que debe evitar, los cuales pueden representarse con polígonos o modelos tridimensionales.

1.5.3. ¿Qué algoritmo de geometría computacional podría utilizarse?

- **Agricultura de precisión:** Se utilizaría la búsqueda de vecinos más cercanos para analizar los puntos obtenidos con el sensor LiDAR. Se identifican los puntos aislados que pueden ser errores de medición, eliminar ruido y calcular la orientación de las superficies de las

plantas y del suelo. Se podrían utilizar métodos de triangulación para conectar los puntos y generar una representación tridimensional de los cultivos y del terreno.

- **Diseño mecánico (CAD):** Se utilizaría la detección de intersecciones (intersection detection) para identificar los puntos donde dos o más piezas tridimensionales se cruzan. Esta información permitiría calcular cómo deben modificarse las superficies de los objetos cuando se realizan operaciones como unir dos piezas, perforar una superficie o quitar material.
- **Ingeniería civil:** Se utilizaría la triangulación de Delaunay para conectar los puntos obtenidos con mediciones topográficas, teniendo en cuenta tanto su posición como la altura del terreno. Esto permitiría formar una red de triángulos que representa la superficie del terreno.
- **Robótica:** Se utilizarían los diagramas de Voronoi para encontrar rutas que no permitan que el robot colisione contra obstáculos. Esto permitiría planificar recorridos más seguros, evitando pasar demasiado cerca de paredes, objetos u otras estructuras. Además, se utilizaría la envolvente convexa (convex hull) para representar de manera simplificada la forma exterior del robot y la detección de intersecciones (Intersection Detection) para comprobar si su trayectoria o alguna de sus partes puede chocar con los obstáculos.

1.5.4. ¿Qué paquete de CGAL permitiría abordar el problema?

- **Agricultura de precisión:** Se utilizarían los paquetes Point Set Processing y Poisson Surface Reconstruction. El primero permite procesar las nubes de puntos obtenidas con sensores LiDAR o drones, eliminando mediciones incorrectas, reduciendo la cantidad de puntos y calculando la orientación de las superficies. El segundo permite utilizar esos datos para reconstruir una superficie tridimensional que represente los cultivos y el terreno.
- **Diseño mecánico (CAD):** Se utilizaría el paquete 3D Boolean Operations on Nef Polyhedra, que permite realizar operaciones entre objetos tridimensionales, como unir dos piezas, encontrar la parte que comparten o quitar una parte de un objeto. Por ejemplo, se podría utilizar para crear una perforación en una pieza o combinar diferentes figuras para obtener un modelo más complejo.
- **Ingeniería civil:** Se utilizaría el paquete 2D Triangulations, para conectar los puntos obtenidos con mediciones topográficas y formar una red de triángulos que represente la superficie del terreno. También permite incluir líneas que representan características importantes del relieve, como ríos y bordes. Para simulaciones que necesitan volúmenes tridimensionales, se podría complementar con 3D Mesh Generation, que permite dividir un volumen en elementos más pequeños para realizar análisis y simulaciones de ingeniería.
- **Robótica:** Se podrían utilizar los paquetes 2D Arrangements y AABB Tree. El primero permite representar los obstáculos con líneas y curvas, dividiendo el espacio en diferentes regiones para identificar dónde se encuentran los obstáculos y dónde hay espacio libre. El segundo permite detectar si una trayectoria o alguna parte del robot se intersecta con los obstáculos de su entorno.

1.5.5. ¿Qué dificultades computacionales y numéricas deberían considerarse?

Agricultura de precisión:

- Dificultades computacionales: Los sensores LiDAR instalados en drones pueden generar muchos puntos que representan el terreno y los cultivos. Procesar toda esta información requiere

bastante memoria y capacidad de procesamiento. Es necesario utilizar estructuras de datos que permitan organizar los puntos y encontrar rápidamente los que están más cerca, como los árboles k-d.

- Dificultades numéricas: Las mediciones realizadas por los sensores pueden contener errores o variaciones que no corresponden a la superficie real. Esto puede dificultar el cálculo de la orientación de las superficies con los vectores normales.

Diseño mecánico (CAD):

- Dificultades numéricas: Al realizar operaciones entre piezas tridimensionales, como unirlas, cortarlas o quitar partes de ellas, pueden aparecer errores de redondeo en los cálculos. Estos errores pueden acumularse cuando se realizan muchas operaciones. Se pueden generar modelos con pequeños espacios o caras desalineadas.

- Dificultades computacionales: Para evitar estos problemas, especialmente cuando las piezas tienen caras alineadas, superficies que coinciden o aristas superpuestas, se pueden necesitar métodos de cálculo exacto, pero, estos métodos requieren más memoria y tiempo de procesamiento que los cálculos aproximados. Por ejemplo, utilizar Exact_predicates_exact_constructions_kernel permite mantener la exactitud tanto en las decisiones geométricas como en la generación de nuevas coordenadas, pero aumenta el costo computacional.

Ingeniería civil:

- Dificultades numéricas: Los datos topográficos pueden contener puntos que se encuentran sobre una misma línea, como ocurre al realizar mediciones a lo largo de una carretera. También pueden presentarse puntos que forman configuraciones geométricas especiales. Si los cálculos se realizan con precisión insuficiente, los errores de redondeo pueden hacer que el algoritmo interprete incorrectamente la posición de los puntos y genere triángulos mal conectados.

- Dificultades computacionales: Al construir una malla del terreno, es necesario respetar líneas que representan características importantes, como carreteras, ríos y bordes. Estas líneas deben incorporarse correctamente a la triangulación para que el modelo represente la forma real del terreno.

Robótica:

- Dificultades numéricas: Los algoritmos de robótica deben determinar con precisión si una trayectoria está libre o si atraviesa algún obstáculo. Los errores de redondeo pueden provocar que el sistema interprete incorrectamente la posición de los objetos o los puntos de intersección. Esto puede hacer que el robot considere que existe un obstáculo cuando no lo hay o, que no detecte una colisión que realmente podría ocurrir.

- Dificultades computacionales: Los robots y drones necesitan calcular distancias, detectar obstáculos y actualizar sus trayectorias mientras están en movimiento. Estas operaciones deben realizarse rápidamente para que el sistema pueda reaccionar a tiempo, es necesario encontrar un equilibrio entre la rapidez y la precisión de los cálculos. Una alternativa es utilizar kernels como Exact_predicates_inexact_constructions_kernel.

1.5.6. Reto de integración

- Problema propuesto: Obtener modelos tridimensionales de los cultivos y del terreno utilizando información recopilada con drones equipados con sensores LiDAR.

Estos datos pueden contener errores, puntos aislados y zonas donde faltan mediciones. Por esta razón, es necesario procesarlos para eliminar el ruido y construir un modelo tridimensional que represente adecuadamente el terreno y los cultivos.

- Entradas:

- **Nube de puntos tridimensional:** Conjunto de coordenadas X, Y y Z obtenidas con el sensor LiDAR instalado en el dron. Estos datos pueden almacenarse en archivos como .las, .ply o .csv y contienen muchos puntos que representan el terreno y la vegetación.
- **Parámetros de configuración:** Son los valores que permiten controlar cómo se procesarán los datos. Se deben definir los criterios para identificar y eliminar puntos incorrectos, la cantidad de vecinos que se utilizarán para analizar cada punto y el nivel de detalle que tendrá el modelo tridimensional final.

- Algoritmos y uso de CGAL:

Primero se procesan los puntos obtenidos con el LiDAR para eliminar aquellos que se encuentran demasiado alejados de sus vecinos y que podrían corresponder a errores de medición. Para esto se utiliza la búsqueda de vecinos más cercanos como los árboles k-d. También se puede reducir la cantidad de puntos para facilitar los cálculos posteriores.

Después se calculan los vectores normales de los puntos. Estos vectores indican la dirección perpendicular a la superficie en cada posición. Esta información es importante para conocer la orientación del terreno y de las hojas, y es necesaria para algunos métodos de reconstrucción tridimensional, como el método de Poisson.

Después de procesar los datos, se utilizan algoritmos de reconstrucción de superficies, como el método de Poisson, para conectar los puntos y generar una superficie tridimensional formada por vértices, aristas y caras. También se puede utilizar la triangulación para representar la superficie con triángulos. Durante este proceso se puede utilizar un Geometry Kernel como Exact_predicates_inexact_constructions_kernel, que permite tomar decisiones geométricas exactas y realizar cálculos aproximados al generar nuevas coordenadas.

- Resultados esperados:

Se espera obtener un modelo tridimensional de los cultivos y del terreno, representado con una malla formada por puntos, líneas y superficies triangulares.

Este modelo permitiría observar la forma del terreno, identificar diferencias de altura, analizar la distribución de las plantas y calcular características como el volumen de la vegetación.

También se espera que la malla tenga una buena calidad geométrica, evitando agujeros, superficies mal conectadas y elementos que se crucen incorrectamente.

- Métricas de evaluación:

Se revisaría si el modelo tridimensional tiene agujeros, superficies mal conectadas o intersecciones incorrectas.

Se puede comparar la posición de los puntos originales del LiDAR con la superficie reconstruida para medir qué tan cerca se encuentra el modelo de las mediciones reales usando el error cuadrático medio. Un valor más bajo indicaría que la superficie reconstruida representa mejor los datos originales.

Se mediría cuánto tiempo necesita el algoritmo para procesar la nube de puntos y construir el modelo tridimensional.

2 Actividad práctica: nube de puntos, envolvente convexa y triangulación de Delaunay

2.1 Procedimiento

1. Se creó un archivo llamado **ejercicio_practico.cpp**, que contiene el código proporcionado en la guía.

2. Se creó el archivo **CMakeLists.txt** para poder construir y ejecutar el código en **ejercicio_practico.cpp**.

3. Desde la terminal de Ubuntu en la carpeta del proyecto, se ejecuto el comando **cmake -S . -B build** seguido de **cmake - -build build**. Estos comandos permiten generar los archivos de construcción y para compilar y enlazar el programa **ejercicio_practico.cpp** con su ejecutable **taller_cgal**.

4. Se puede ejecutar el código con el siguiente comando con unos parámetros ejemplo **./build/taller_cgal 100 42**. Estos parámetros corresponden a la cantidad de puntos a generar y la semilla para el generador de números pseudoaleatorios respectivamente.

5. Se cambió la cantidad de puntos a generar y la semilla de generación, realizando 4 ejecuciones para: 100 puntos con semilla 42, 500 puntos con semilla 120, 1000 puntos con semilla 70 y 5000 puntos con semilla 30.

6. Usando un código de Python, se graficaron los puntos generados en el item anterior.

El código en C++ en el archivo **ejercicio_practico.cpp** utilizado para el experimento, es el siguiente

```cpp
#include <CGAL/Exact_predicates_inexact_constructions_kernel.h>
#include <CGAL/convex_hull_2.h>
#include <CGAL/Delaunay_triangulation_2.h>

#include <iterator>
#include <algorithm>
#include <iostream>
#include <random>
#include <vector>
#include <chrono> // Para medir el tiempo de ejecución

using Kernel = CGAL::Exact_predicates_inexact_constructions_kernel;
using Point = Kernel::Point_2;
using Delaunay = CGAL::Delaunay_triangulation_2<Kernel>;

int main(int argc, char* argv[])
{
    // Numero de puntos y semilla opcionales desde la linea de comandos.
    const std::size_t n = (argc > 1) ? static_cast<std::size_t>(std::stoul(argv[1])) : 100;
    const unsigned int seed = (argc > 2) ? static_cast<unsigned int>(std::stoul(argv[2])) : 42u;

    // Generador reproducible de coordenadas en [0, 100].
    std::mt19937 generator(seed);
    std::uniform_real_distribution<double> distribution(0.0, 100.0);

```

```
    std::vector<Point> points;
    points.reserve(n);

    for (std::size_t i = 0; i < n; ++i) {
        points.emplace_back(distribution(generator), distribution(generator));
    }

    // Calculo de la envolvente convexa.
    auto start_hull = std::chrono::high_resolution_clock::now(); // Se empieza a medir el tiempo de ejecución de la
     envolvente convexa
    std::vector<Point> hull;
    CGAL::convex_hull_2(points.begin(), points.end(), std::back_inserter(hull));
    auto end_hull = std::chrono::high_resolution_clock::now();
    auto duration_hull = std::chrono::duration_cast<std::chrono::microseconds>(end_hull - start_hull).count();

    // Construccion de la triangulacion de Delaunay.
    Delaunay dt;
    auto start_delaunay = std::chrono::high_resolution_clock::now(); // Se empieza a medir el tiempo de ejecución de la
      triangulación de Delaunay
    dt.insert(points.begin(), points.end());
    auto end_delaunay = std::chrono::high_resolution_clock::now();
    auto duration_delaunay = std::chrono::duration_cast<std::chrono::microseconds>(end_delaunay - start_delaunay).count
     ();

    // Resumen de resultados.
    std::cout << "Puntos generados: " << points.size() << '\n';
    std::cout << "Vertices de la envolvente: " << hull.size() << '\n';
    std::cout << "Vertices distintos en Delaunay: " << dt.number_of_vertices() << '\n';
    std::cout << "Aristas finitas de Delaunay: " << std::distance(dt.finite_edges_begin(), dt.finite_edges_end()) << '\
     n';
    std::cout << "Caras finitas de Delaunay: " << dt.number_of_faces() << '\n';
    std::cout << "Tiempo Envolvente Convexa: " << duration_hull << " microsegundos\n";
    std::cout << "Tiempo Triangulacion Delaunay: " << duration_delaunay << " microsegundos\n";

    // Exportar puntos originales.
    std::cout << "\n# points: x y\n";
    for (const auto& p : points) {
        std::cout << p.x() << " " << p.y() << '\n';
    }

    // Exportar vertices de la envolvente en orden ciclico.
    std::cout << "\n# convex_hull: x y\n";
    for (const auto& p : hull) {
        std::cout << p.x() << " " << p.y() << '\n';
    }

    // Exportar aristas finitas de la triangulacion.
    std::cout << "\n# delaunay_edges: x1 y1 x2 y2\n";
    for (auto edge = dt.finite_edges_begin(); edge != dt.finite_edges_end(); ++edge) {
        auto face = edge->first;
        int index = edge->second;
        Point p = face->vertex((index + 1) % 3)->point();
        Point q = face->vertex((index + 2) % 3)->point();
        std::cout << p.x() << " " << p.y() << " " << q.x() << " " << q.y() << '\n';
    }

    return 0;
}
```

Listing 1: Ejemplo de inserción de puntos en CGAL.

Este código tiene un generador de números pseudoaleatorios controlado por una semilla y una cantidad definida en los parámetros del comando de ejecución mencionado en el procedimiento, creando una nube bidimensional de puntos dentro de un área de 100x100 unidades. Posteriormente, procesa estas coordenadas de entrada utilizando la biblioteca CGAL para calcular dos representaciones geométricas: la envolvente convexa, que determina el polígono mínimo que encierra todos los puntos, y la triangulación de Delaunay, que conecta todos los elementos en una red de triángulos insertándolos en una estructura de datos especializada. Una vez procesada la geometría, el programa extrae los resultados e imprime los vértices de la envolvente y el número de vértices, aristas y caras finitas de la triangulación. Por último, itera sobre las estructuras generadas para exportar en formato de texto plano las coordenadas de los puntos originales, los vértices de la frontera en orden cíclico y los pares de coordenadas correspondientes a cada arista triangular.

Además, se hizo un código de Python (qué es el archivo **graficador_datos.py**) para generar las gráficas de los puntos y su envolvente convexa, como además, calcular la cantidad de puntos en la frontera de la envolvente y la cantidad de puntos dentro de la misma. El código se muestra a continuación:

```
import matplotlib.pyplot as plt
import matplotlib.pyplot as plt
```

```
import sys
def graficar_datos_cgal(nombre_archivo):
    puntos_x, puntos_y = [], [] # Vectores para almacenar las coordenadas de los puntos
    hull_x, hull_y = [], [] # Vectores para almacenar las coordenadas de la envolvente convexa
    aristas = [] # Vector para almacenar la cantidad de aristas de la triangulación de Delaunay

    seccion_actual = None

    # Leer el archivo
    try:
        with open(nombre_archivo, 'r') as f:
            for linea in f:
                linea = linea.strip()
                if not linea: continue

                # Detectar en qué sección de datos estamos para dividir correctamente los mismos
                if linea.startswith('# points:'):
                    seccion_actual = 'puntos'
                    continue
                elif linea.startswith('# convex_hull:'):
                    seccion_actual = 'hull'
                    continue
                elif linea.startswith('# delaunay_edges:'):
                    seccion_actual = 'aristas'
                    continue
                # Se ignora el resumen de texto inicial
                elif linea.startswith('Puntos') or linea.startswith('Vertices') or linea.startswith('Aristas') or linea
     .startswith('Caras'):
                    continue

                # Extraer las coordenadas
                partes = linea.split()
                if seccion_actual == 'puntos' and len(partes) == 2:
                    puntos_x.append(float(partes[0]))
                    puntos_y.append(float(partes[1]))
                elif seccion_actual == 'hull' and len(partes) == 2:
                    hull_x.append(float(partes[0]))
                    hull_y.append(float(partes[1]))
                elif seccion_actual == 'aristas' and len(partes) == 4:
                    aristas.append((float(partes[0]), float(partes[1]), float(partes[2]), float(partes[3])))
    except FileNotFoundError:
        print(f"Error: No se pudo encontrar el archivo '{nombre_archivo}'.")
        return

    # Cerrar el polígono de la envolvente conectando el último punto con el primero
    if hull_x:
        hull_x.append(hull_x[0])
        hull_y.append(hull_y[0])

    # Calcular la cantidad de puntos en la frontera restando el punto duplicado
    puntos_frontera = len(hull_x) - 1
    print(f"Puntos en la frontera de la envolvente: {puntos_frontera}")
    print(f"Puntos en el interior: {len(puntos_x) - puntos_frontera}")

    # Configurar la gráfica
    plt.figure(figsize=(10, 10))

    # Se grafican las aristas de la triangulación de Delaunay
    for x1, y1, x2, y2 in aristas:
        plt.plot([x1, x2], [y1, y2], color='lightgray', linewidth=0.8, zorder=1)

    # Se grafica la envolvente convexa
    plt.plot(hull_x, hull_y, color='red', linewidth=2, label='Envolvente Convexa', zorder=2)

    # Se grafica la nube de puntos
    plt.scatter(puntos_x, puntos_y, color='blue', s=10, label='Nube de Puntos', zorder=3)

    plt.title('Triangulación de Delaunay y Envolvente Convexa')
    plt.xlabel('X')
    plt.ylabel('Y')
    plt.legend()
    plt.axis('equal') # Se mantiene la proporción geométrica 1 a 1
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.show()

def main():
    # Toma el archivo desde la consola, o usa uno por defecto si no se especifica
    if len(sys.argv) > 1:
        archivo = sys.argv[1]
    else:
        archivo = '100_puntos_semilla_42.txt'

    print(f"Generando gráfica")
    graficar_datos_cgal(archivo)


main()
```

Listing 2: Script de graficación de datos graficador$_datos.py$

Con este código, se pudieron visualizar gráficamente los datos obtenidos con las 4 ejecuciones mencionadas anteriormente:

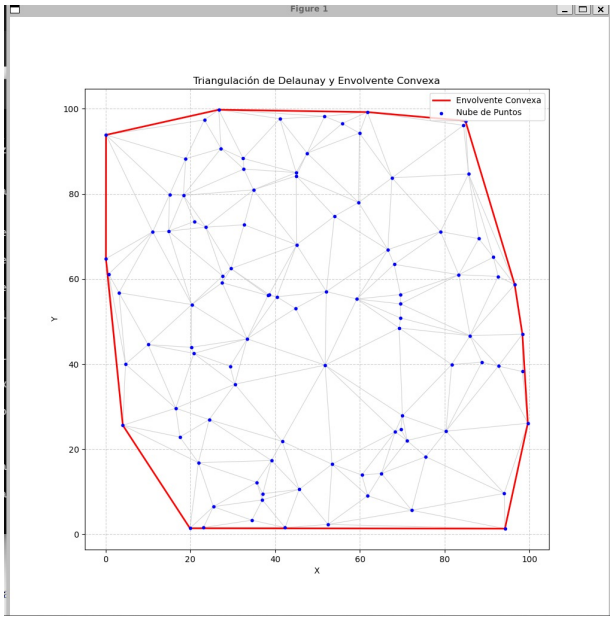


Figura 2: Gráfica con 100 puntos usando la semilla 42

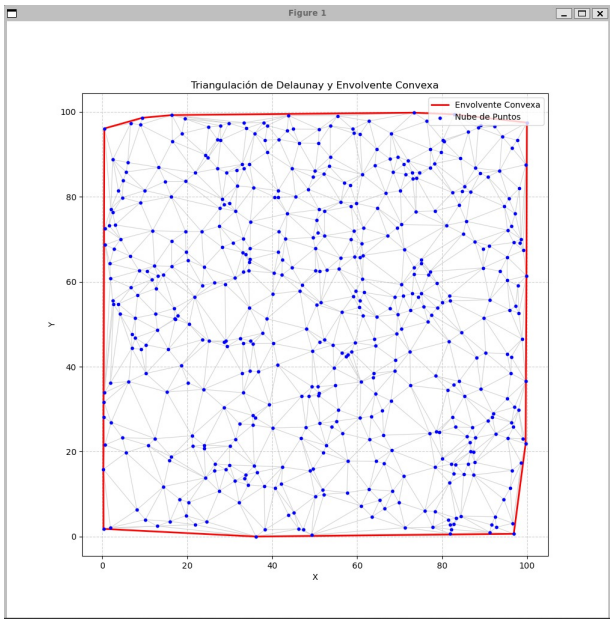


Figura 3: Gráfica con 500 puntos usando la semilla 120

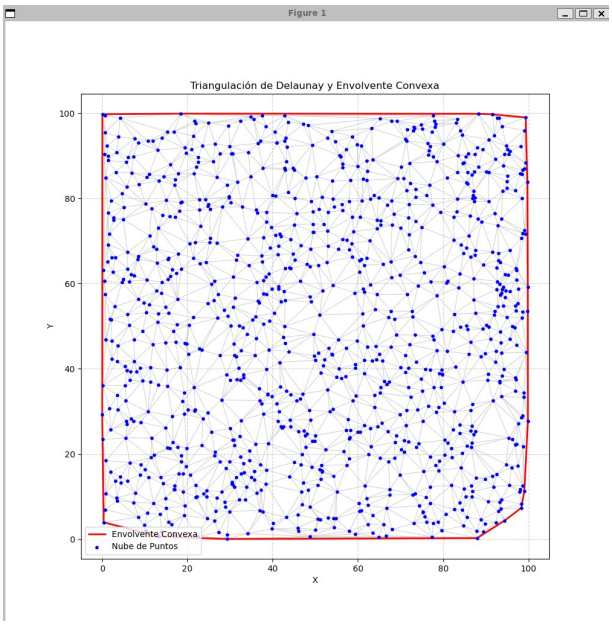


Figura 4: Gráfica con 1000 puntos usando la semilla 70

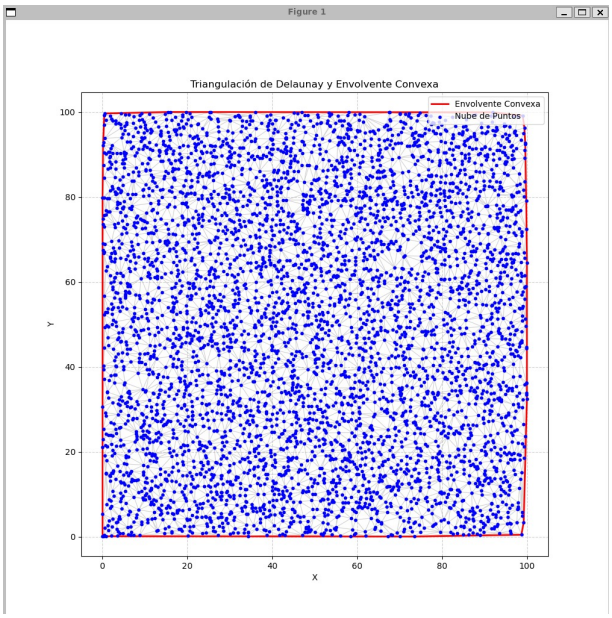


Figura 5: Gráfica con 5000 puntos usando la semilla 30

2.2 Preguntas de análisis de resultados

2.2.1. ¿Qué proporción de los puntos generados pertenece a la frontera de la envolvente convexa? ¿Cómo cambia al aumentar el número de puntos?

Se evaluó en cada caso la cantidad de puntos que quedaban sobre la frontera de la envolvente y se obtuvo lo siguiente:

- **Con 100 puntos y semilla 42:** Ejecutando el script de Python, se obtiene los siguientes puntos en la frontera y en el interior:

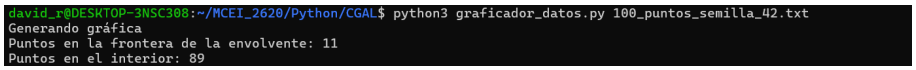


Figura 6: Cantidad de puntos en la frontera si son generados 100 puntos con la semilla 42

Esto corresponde a aproximadamente a $\frac{1}{10}$ de la cantidad total de 100 puntos pseudoaleatorios generados.

- **Con 100 puntos y semilla 120:** En este caso se obtuvo lo siguiente:



Figura 7: Cantidad de puntos en la frontera si son generados 500 puntos con la semilla 120

Esto corresponde a aproximadamente a $\frac{1}{50}$ de la cantidad total de 500 puntos generados.

- **Con 1000 puntos y semilla 70:** En este caso se obtuvo lo siguiente:

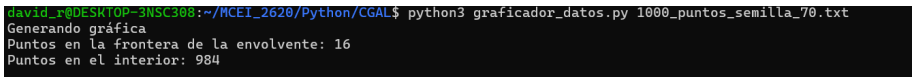


Figura 8: Cantidad de puntos en la frontera si son generados 1000 puntos con la semilla 70

Esta cantidad de puntos en la frontera corresponde a $\frac{16}{1000} \approx \frac{1}{50}$ del total de los 1000 puntos generados.

- **Con 5000 puntos y semilla 30:** Se obtuvo lo siguiente:

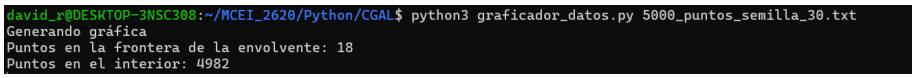


Figura 9: Cantidad de puntos en la frontera si son generados 5000 puntos con la semilla 30

Esta cantidad de puntos en la frontera corresponde a $\frac{18}{5000} \approx \frac{1}{250}$ del total de los 1000 puntos generados.

De lo mostrado anteriormente, puede verse que la cantidad de puntos en la frontera estuvo entre 11 y 18 indepndientemente de la cantidad de puntos y de la semilla. Lo que si se pudo observar es que a medida que se aumentaron los puntos generados, aumentaba muy poco la cantidad de puntos en la frontera, y que las proporción disminuyó desde $\frac{11}{100} = 0{,}11$ a $\frac{18}{5000} = 0{,}0036$, debido a que casi no cambiaba la cantidad de puntos en la frontera y si aumentaban considerablemente los puntos dentro de la envolvente.

2.2.2. ¿Por qué la envolvente convexa no contiene información sobre la conectividad de los puntos interiores?

La envolvente convexa se define como el menor conjunto convexo que contiene todos los puntos, limitándose únicamente a describir el contorno perimetral que encierra a los puntos. Por definición, los puntos interiores no forman parte de su frontera y no se establece ninguna relación geométrica entre ellos. La envolvente convexa solo depende de los puntos extremos que conforman a la misma.

2.2.3. ¿Qué información adicional aporta la triangulación de Delaunay?

La triangulación de Delaunay cumple que para cada triángulo generado, su circunferencia no contenga más de 3 vértices. Para garantizar esto, aporta información sobre la proximidad espacial y topología interna, lo que puede evidenciarse con:

- **Conectividad:** Cada punto queda unido a sus vecinos, y las aristas indican proximidad a los mismos.
- **Propiedad del círculo vacío:** Tiende a evitar triángulos muy delgados y largos, porque maximiza el ángulo mínimo, útil para interpolación y mallado.
- **La envolvente:** La frontera exterior de la triangulación es la envolvente convexa.

2.2.4. ¿Cuántos vértices, aristas y caras finitas aparecen para cada tamaño de nube? ¿Los resultados concuerdan con las relaciones de Euler para una triangulación plana?

Para cada una de las cuatro ejecuciones realizadas, se obtuvieron las siguientes cantidades de vértices, aristas y caras finitas:

```
david_r@DESKTOP-3NSC308:~/MCEI_2620/C_C++/CGAL$ ./build/taller_cgal 100 42
Puntos generados: 100
Vertices de la envolvente: 11
Vertices distintos en Delaunay: 100
Aristas finitas de Delaunay: 286
Caras finitas de Delaunay: 187

# points: x y
18.3435 79.6543
59.685 77.9691
9.99749 44.5833
33.3709 45.9249
65.0888 14.2867
72.1999 5.64116
0.0778765 93.8553
61.7482 99.2212
0.706631 61.1653
52.4775 2.30624
4.66657 39.9861
23.2771 97.3756
61.8386 9.06064
98.3231 38.2462
```

Figura 10: Cantidades con 100 puntos y semilla 42

```
david_r@DESKTOP-3NSC308:~/MCEI_2620/C_C++/CGAL$ ./build/taller_cgal 500 120
Puntos generados: 500
Vertices de la envolvente: 12
Vertices distintos en Delaunay: 500
Aristas finitas de Delaunay: 1485
Caras finitas de Delaunay: 986

# points: x y
99.688 36.555
59.036 56.5525
95.9299 53.3673
86.1689 95.2494
49.5669 15.9076
46.3308 92.6601
1.61814 73.2051
40.2244 45.4868
34.9125 46.0359
```

Figura 11: Cantidades con 500 puntos y semilla 120



Figura 12: Cantidades con 1000 puntos y semilla 70

Para poder visualizar correctamente los resultados con 5000 puntos, se tuvo que almacenar directamente la salida de la términal en un archivo de texto empleando el comando **./build/taller_cgal 5000 30 > 5000_puntos_semilla_30.txt**:

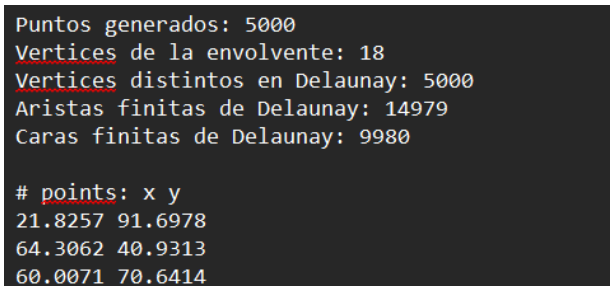


Figura 13: Cantidades con 1000 puntos y semilla 70

Los resultados anteriores, pueden consignarse en una tabla:

Cuadro 1: Propiedades topológicas (vértices, aristas y caras) generadas con semillas aleatorias.

Nube de Puntos (n)	**Vértices en Frontera** (h)	**Aristas Finitas** (E)	**Caras Finitas** (F)
100	11	286	187
500	12	1,485	986
1,000	16	2,981	1,982
5,000	18	14,979	9,980

Conociendo la cantidad de caras de Delaunay, aristas de Delaunay y vértices distintos de Delaunay (que también es la cantidad total de puntos generados), se puede comprobar la relación de Euler:

$$Vertices_{Delaunay} - Aristas_{Delaunay} + Caras_{Delaunay} = 1 \tag{1}$$

Debido a que el código en el archivo **ejercicio_practico.cpp** ahorra calcular la cantidad de caras de Delaunay y la cantidad de aristas de Delaunay, se puede comprobar para cada una de las 4 nubes generadas, la relación de Euler:

- **Con 100 puntos y semilla 42:** Usando los datos de la imagen 10, se obtiene que se cumple la relación de Euler, debido a que el resultado es 1:

$$100 - 186 + 187 = 1$$

- **Con 500 puntos y semilla 120:** Usando los datos de la imagen 11, se cumple la relación de Euler:

$$500 - 1485 + 986 = 1 \tag{2}$$

- **Con 1000 puntos y semilla 70:** Con los datos de la imagen 12, se cumple la relación de Euler:

$$1000 - 2981 + 1982 = 1 \tag{3}$$

- **Con 5000 puntos y semilla 30:** Con los datos de la imagen 13, también se cumple en este caso la relación de Euler:

$$5000 - 14979 + 9980 = 1 \tag{4}$$

2.2.5. ¿Cómo cambia el tiempo de ejecución al aumentar el número de puntos? ¿Qué factores pueden afectar la medición?

Se modificó el código en C++ para poder medir los tiempos del cálculo de la envolvente convexa y el tiempo que tarda en hacerse la triangulación Delaunay:

```
david_r@DESKTOP-3NSC308:~/MCEI_2620/C_C++/CGAL$ ./build/taller_cgal 100 42
Puntos generados: 100
Vertices de la envolvente: 11
Vertices distintos en Delaunay: 100
Aristas finitas de Delaunay: 286
Caras finitas de Delaunay: 187
Tiempo Envolvente Convexa: 315 microsegundos
Tiempo Triangulacion Delaunay: 2413 microsegundos
```

Figura 14: Tiempos de ejecución con 100 puntos y semilla 42

```
david_r@DESKTOP-3NSC308:~/MCEI_2620/C_C++/CGAL$ ./build/taller_cgal 500 120
Puntos generados: 500
Vertices de la envolvente: 12
Vertices distintos en Delaunay: 500
Aristas finitas de Delaunay: 1485
Caras finitas de Delaunay: 986
Tiempo Envolvente Convexa: 1125 microsegundos
Tiempo Triangulacion Delaunay: 8513 microsegundos
```

Figura 15: Tiempos de ejecución con 500 puntos y semilla 120

```
david_r@DESKTOP-3NSC308:~/MCEI_2620/C_C++/CGAL$ ./build/taller_cgal 1000 70
Puntos generados: 1000
Vertices de la envolvente: 16
Vertices distintos en Delaunay: 1000
Aristas finitas de Delaunay: 2981
Caras finitas de Delaunay: 1982
Tiempo Envolvente Convexa: 1814 microsegundos
Tiempo Triangulacion Delaunay: 15572 microsegundos
```

Figura 16: Tiempos de ejecución con 1000 puntos y semilla 70

```
  GNU nano 7.2                                    5000_puntos_semilla_3
Puntos generados: 5000
Vertices de la envolvente: 18
Vertices distintos en Delaunay: 5000
Aristas finitas de Delaunay: 14979
Caras finitas de Delaunay: 9980
Tiempo Envolvente Convexa: 12348 microsegundos
Tiempo Triangulacion Delaunay: 121946 microsegundos
```

Figura 17: Tiempos de ejecución con 5000 puntos y semilla 30

Se pueden consignar los resultados de los tiempos en una tabla:
A medida que se aumentó la cantidad de puntos, también aumentó el tiempo de ejecución tanto para el calculo de la envolvente, como para la triangulación.

Algunos factores que pueden afectar la medición son:

Cuadro 2: Tiempos de ejecución para la Envolvente Convexa y Triangulación de Delaunay según el tamaño de la nube de puntos.

Nube de Puntos (n)	**Tiempo Envolvente** (μs)	**Tiempo Delaunay** (μs)
100	315	2,413
500	1,125	8,513
1,000	1,814	15,572
5,000	12,348	121,946

- Sobrecarga del sistema operativo y virtualización, porque Al ejecutar el código en Ubuntu a través de WSL, el procesador comparte recursos continuamente con los procesos nativos de Windows en segundo plano, lo que hace que si el planificador del sistema operativo interrumpe el algoritmo de CGAL por una fracción de milisegundo para atender otra tarea, el reloj de la librería chrono sigue corriendo, registrando un tiempo total superior al tiempo real de procesamiento del procesador.
- Asignación dinámica de memoria porque al agregar los puntos uno por uno para construir la triangulación de Delaunay, la estructura de datos necesita solicitar y asignar nuevos espacios en la memoria RAM durante la ejecución del programa. Este proceso depende de la gestión de memoria del sistema operativo, que puede tardar diferentes intervalos de tiempo en encontrar y reservar los bloques disponibles. Por esta razón, el tiempo de ejecución puede presentar pequeñas variaciones entre una ejecución y otra.

2.2.6. ¿Qué ocurre si se introducen puntos repetidos, colineales o cocirculares? Diseñe experimentos para estudiar estos casos.

1. **Puntos repetidos** En este experimento, en lugar de generar datos pseudoaleatorios como se estaba haciendo anteriormente con el programa **ejercicio_practico.cpp**, se debe generar el mismo punto **n** veces.

   El experimento tiene el propósito de evaluar el comportamiento del algoritmo que calcula la envolvente convexa y el algoritmo de triangulación de Delaunay, como también, evaluar la robustez de las estructuras de datos de CGAL cuando se da un caso extremo de puntos repetidos.

   - En este caso, todos los **n** puntos generados son exactamente iguales. Por lo que el código **ejercicio_practico.cpp** fue modificado cambiando el bucle que generaba los puntos aleatorios, por un bucle que genere el mismo punto **n** veces.
   - Este experimento ya no requiere una semilla para generar datos aleatorios, únicamente se requiere la cantidad de puntos a generar **n**.
   - Se evalúa si la estructura de la triangulación inserta los **n** puntos en la memoria o si el árbol de búsqueda de CGAL identifica y descarta las coordenadas duplicadas durante la fase de inserción (dt.insert()).
   - Al tener un conjunto de **n** puntos superpuestos, esto constituye un único vértice en el plano. Entonces, el experimento verifica si el algoritmo devuelve un polígono de envolvente convexa y una malla de Delaunay compuestos por exactamente 1 vértice, 0 aristas y 0 caras.

   Los resultados con 100 puntos generados en exactamente la mitad del cuadrado 100x100 son los siguientes:

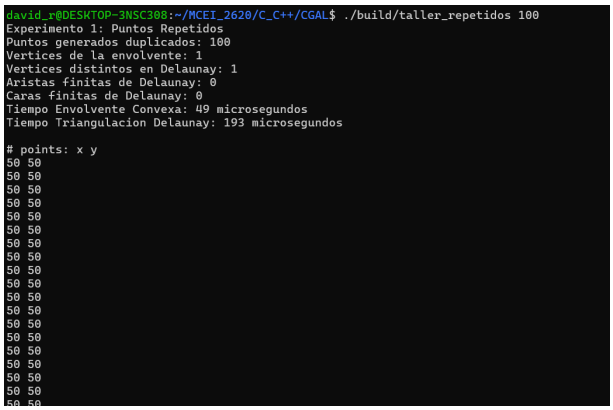


Figura 18: Valores resultantes del experimento con puntos repetidos

Se pudo comprobar que CGAL reconoce y elimina las coordenadas repetidas durante el proceso de inserción. Aunque se ingresaron 100 puntos, la estructura CGAL::Delaunay_triangulation_2 almacenó únicamente un vértice, como se evidencia en el resultado de que el número de vértices es 1. Gráficamente así se ven los puntos:

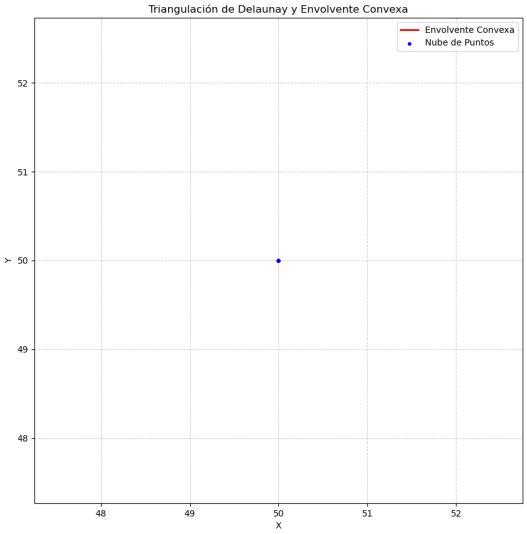


Figura 19: Gráfica con puntos repetidos

La gráfica obtenida permite comprobar visualmente los resultados registrados en la consola de Ubuntu. Se observa un único punto azul ubicado en las coordenadas X=50 y Y=50, que representa la posición de los 100 puntos ingresados. Se pueden observar dos cosas más:

- No hay envolvente, lo que es geométricamente correcto, ya que la envolvente convexa de un único punto es el mismo punto y carece de perímetro o área.
- Como la consola reporta cero aristas, tampoco es posible representar una malla de triángulos. Por lo tanto, la gráfica muestra únicamente el punto correspondiente a las coordenadas ingresadas.

De esto se puede concluir que, el kernel Exact_predicates_inexact_constructions_kernel demuestra total robustez geométrica: en lugar de fallar al intentar calcular áreas con área

nula o arrojar divisiones por cero, el algoritmo simplifica el caso extremo de tal modo que sea seguro.

2. **Puntos Colineales:** Los puntos generados deben pertenecer todos a una misma linea recta para considerarse colineales, la idea de este experimento es evaluar la robustez de CGAL ante este caso extremo.

   - Se generan **n** puntos de tal modo que todos pertenezcan a una línea recta horizontal.
   - La idea es ver que el polígono convexo más pequeño que puede contener un segmento de línea es el propio segmento de línea. Además que la envolvente no será cerrada, ni va a contener en su interior ningún punto.
   - Al no haber puntos superpuestos porque cada punto tiene una coordenada X distinta, se va a detectar una determinada cantidad de vértices **n**, una cantidad de aristas **n-1** y no va a detectar caras porque no se pueden generar triángulos porque todos los puntos son colineales.

   Los resultados del experimento con 100 puntos son los siguientes:

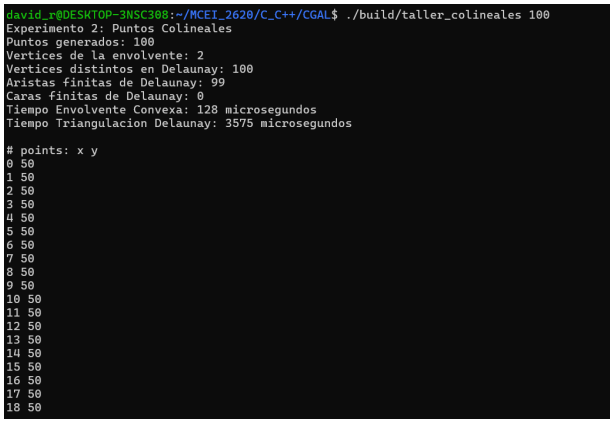


Figura 20: Valores resultantes del experimento con puntos colineales

El algoritmo identificó correctamente que, al estar todos los puntos sobre una misma línea, la envolvente convexa corresponde únicamente al segmento que une los dos extremos del conjunto. Por esta razón, de los 100 puntos ingresados, solo se conservaron 2 vértices, correspondientes al primero y al último punto.
En este caso, a diferencia del experimento anterior, las 100 coordenadas son diferentes, por lo que CGAL almacenó los 100 vértices. Como todos se encuentran alineados, la triangulación los conectó consecutivamente mediante 99 aristas finitas. Sin embargo, no se generaron caras triangulares, por lo que el resultado fue de 0 caras finitas.

Esto demuestra que CGAL reconoce la condición de colinealidad sin generar triángulos que realmente no existen. En algunos algoritmos que utilizan cálculos de punto flotante, los errores de redondeo pueden producir pequeñas desviaciones y hacer que aparezcan triángulos con áreas extremadamente pequeñas. En este experimento, el kernel Exact_predicates_inexact_constructions_kernel permitió evaluar correctamente las relaciones geométricas y conservar la colinealidad de los puntos.

Gráficamente, se obtiene lo siguiente:

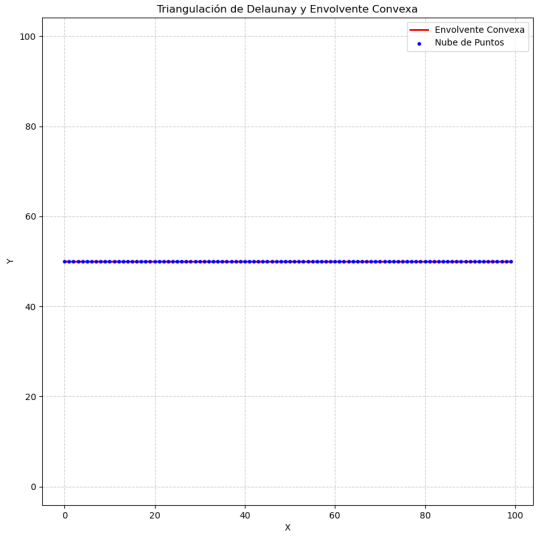


Figura 21: Gráfica con puntos colineales

La gráfica permite observar claramente el comportamiento de los puntos cuando se encuentran alineados dentro de un plano:

- Se observa una línea horizontal formada por los 100 puntos azules, que se extiende desde X=0 hasta X=99, manteniendo la coordenada Y=50.
- La envolvente convexa, representada por una línea roja, coincide con esta misma trayectoria y conecta los dos puntos extremos. Esto permite comprobar visualmente que solo se necesitan dos vértices para representar la envolvente del conjunto.
- No aparece la triangulación de Delaunay. Esto se debe a que no existen tres puntos no colineales que permitan formar una cara triangular, lo cual coincide con los resultados obtenidos en la consola.

3. **Puntos Cocirculares:** Este experimento consiste en generar únicamente puntos que pertenezcan a la misma circunferencia y que tengan la misma distancia a un punto central. La idea es evaluar como ser realizaría la triangulación de Delanuay, si las circunferencias de los triángulos no cumplen con tener exactamente 3 vertices del mismo triángulo formado por 3 puntos. Además, se deben tener en cuenta otras cosas del experimento:

   - Para calcular las coordenadas de los puntos, se utilizan las siguientes expresiones:

   $$P_x = X_{centro} + R \cdot Cos(\theta) \tag{5}$$

   $$P_y = Y_{centro} + R \cdot Sin(\theta) \tag{6}$$

   Donde Px es la coordenada **x** del punto, Py es la coordenada **y**, R es el radio de la circunferencia y θ es el ángulo del segmento entre el centro de la circunferencia y el punto.
   - Desde el punto de vista geométrico, todos los puntos ubicados sobre una circunferencia forman parte de su frontera. Por lo tanto, ninguno queda encerrado dentro de la envolvente formada por los demás.
     Al procesar los puntos, el algoritmo debe reconocer que todos corresponden a vértices de la envolvente convexa.

- Como se mencionó previamente, en este experimento, al seleccionar tres puntos para formar un triángulo, su circunferencia circunscrita coincide con la circunferencia original. Esto significa que los demás puntos también se encuentran sobre el borde de ese mismo círculo. Como resultado, pueden existir diferentes formas de conectar los vértices mediante diagonales sin incumplir la condición de Delaunay. Esta situación se conoce como un caso degenerado de cocircularidad.

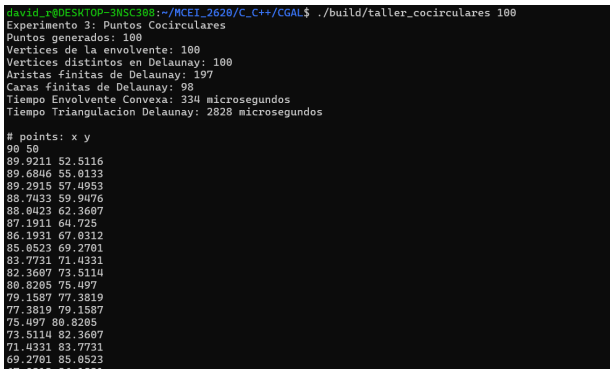


Figura 22: Valores resultantes del experimento con puntos cocirculares

Todos los puntos ubicados sobre el perímetro de la circunferencia forman parte de la envolvente convexa. El algoritmo identificó correctamente esta condición y reportó 100 vértices, sin descartar ninguno de los puntos ingresados. Al conectarlos consecutivamente, se obtiene un polígono regular de 100 lados que representa el contorno del conjunto.
En cuanto a la triangulación de Delanuay, Este caso representa un reto particular para el algoritmo, debido a que todos los puntos se encuentran distribuidos sobre una misma circunferencia. Al seleccionar tres de ellos para formar un triángulo, su circunferencia coincide con la circunferencia original, por lo que los demás puntos quedan ubicados sobre su frontera. Esto significa que existen diferentes formas de conectar los vértices sin incumplir la condición de Delaunay.
A pesar de esta situación, CGAL logró construir la triangulación sin presentar errores. La consola reportó 100 vértices, 197 aristas finitas y 98 caras finitas, lo que indica que se generó una malla completa a partir de los puntos ingresados. Además, estos valores cumplen con la relación de Euler para una triangulación plana de un polígono simple:

$$Vertices_{Delanuay} - Aristas_{Delanuay} + Caras_{Delanuay} = 100 - 197 + 98 = 1$$

Los resultados gráficos son los siguientes:

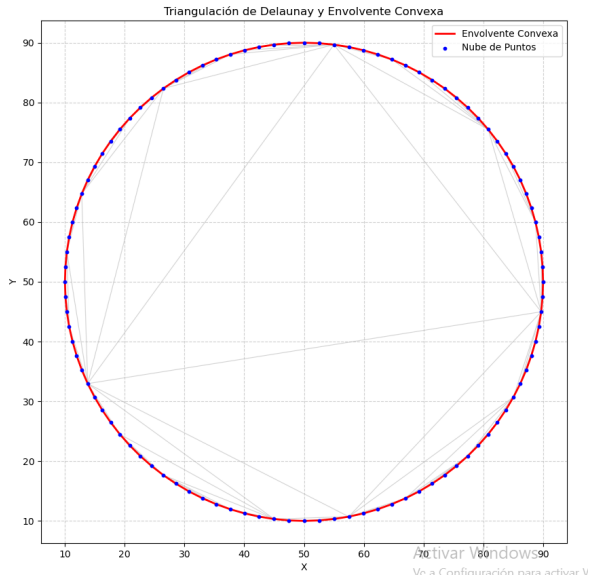


Figura 23: Gráfica con puntos cocirculares

Observando la gráfica obtenida en 23, se puede comprobar que todos los puntos participan en la construcción de la envolvente y que ninguno queda por fuera. Además, en el interior de la circunferencia se observan las líneas grises que representan las aristas de la triangulación de Delaunay. Estas conexiones dividen la región en 98 caras triangulares. También se puede apreciar que CGAL seleccionó una distribución particular de diagonales entre las diferentes configuraciones posibles, obteniendo una triangulación válida del conjunto.

Se debe tener en cuenta algo importante sobre la aproximación de todos los puntos a la circunferencia original, ya las coordenadas de los mismos se calcularon utilizando las funciones std::sin y std::cos, cuyos resultados se almacenan normalmente en variables de tipo double. Debido a la precisión la poca precisión de este tipo de variable, las coordenadas obtenidas pueden presentar pequeñas diferencias respecto a sus posiciones matemáticas ideales sobre la circunferencia. Por esta razón, no se puede asegurar que los 100 puntos sean estrictamente cocirculares desde el punto de vista numérico en todos los casos.

2.2.7. ¿Qué diferencias observa entre ejecutar el mismo experimento con distintas semillas y repetirlo con la misma semilla?

Se realizaron varias ejecuciones para las cantidades de puntos usadas anteriormente, pero todas con exactamente la misma semilla (se eligió 400). Los datos obtenidos en este caso se presentan a continuación en una tabla:

Cuadro 3: Propiedades topológicas (vértices, aristas y caras) generadas con la semilla 400.

Nube de Puntos (n)	**Vértices en Frontera** (h)	**Aristas Finitas** (E)	**Caras Finitas** (F)
100	10	287	188
500	16	1,481	982
1,000	20	2,977	1,978
5,000	26	14,971	9,972

En la tabla anterior, al igual que la tabla 1, muestra la cantidad de vértices de la envolvente

convexa, vértices de Delaunay, aristas finitas de Delaunay y las caras finitas de Delaunay, obtenidas al ejecutar varias veces cada valor con la misma semilla. Los valores se mantuvieron exactamente iguales con cada ejecución.

Las diferencias entre usar semillas aleatorias y usar una semilla fija siempre radican en que cambia la cantidad de vértices de envolvente, las aristas y las caras:

- Para 100 puntos generados, usar la semilla 42 resultó en 11 vértices en la envolvente convexa y 187 caras de Delaunay. Al cambiar a la semilla 400, la geometría cambió a 10 vértices en la envolvente y 188 caras de Delaunay.
- Para 5000, la semilla 120 generó 18 vértices en la envolvente y 9980 caras, mientras que la semilla 400 produjo 26 vértices en la frontera y 9972 caras.

Ocurre lo mismo para los demás casos. Esta variación confirma el comportamiento de la relación de Euler: al cambiar aleatoriamente las posiciones, varía la cantidad de puntos que logran quedar en la frontera de la envolvente, lo que a su vez obliga a que la cantidad de aristas finitas y triángulos internos se ajuste matemáticamente para mantener la triangulación de Delaunay válida.